Software Requirements Specification

Readiness

Version: 1.0
Author: Miguel Ferreira, Miguel Souza, Bernardo Crista, Ana Silva
Date: November 04, 2026
Status: Draft
Instructor: Ricardo Canelas

Revision History

Version	Date	Author	Description of change
0.1	2026-11-04	all	Initial draft
1.0	YYYY-MM-DD	Your Name	First release

Table of Contents

1 Introduction 4
1.1 Purpose 4
1.2 Scope 4
1.3 Definitions, Acronyms, and Abbreviations 4
1.4 References 4
1.5 Overview 4
2 Overall Description 5
2.1 Product Perspective 5
2.2 Product Functions 5
2.3 User Classes and Characteristics 5
2.4 Design and Implementation Constraints 5
2.5 Assumptions and Dependencies 5
3 Specific Requirements 6
3.1 Functional Requirements 6
3.1.1 User Authentication 6
3.1.2 Data Management 6
3.2 External Interface Requirements 7
3.2.1 Software Interfaces 7
3.2.2 Communication Interfaces 7
3.3 Non-Functional Requirements 8
3.3.1 Security 8
3.3.2 Maintainability 8
4 Domain 9
4.1 Entity relationship 9
4.2 User 9
4.3 Role 10
4.4 Book 10
4.5 Review 10
5 Appendices 11
5.1 Appendix A: Use Cases 11
5.2 Appendix B: Open Issues 11

1 Introduction

1.1 Purpose

The following document is presented as the final report for the work developed for C Sharp DEV STREAM 2. Showing the ideas, work and processes used during the development of the project.

1.2 Scope

The software product is called readiness (read + readiness) it is an identical software to Goodreads, and has its basic features to be considered a working MVP. In this case, it allows users to sign in and sign up, it also allows them to add, remove books from their *read* or *want to read* list, it also allows users to add books to a whishlist and more importantly it also allows users to review the books they have read, this in addition to the ability to add the time it took to read a certain book will allow for different dashboards and data analysis.

For the moderation team, the idea is to allow them to add new books that aren't in the platform, not only that it also allows them to moderate the reviews present in a certain book by removing certain bad reviews and finally, they can also approve or disapprove of users requesting author status on the website.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
SRS	Software Requirements Specification
API	Application Programming Interface
UI	User Interface
REST	Representational State Transfer

1.4 References

- IEEE Std 830-1998 / ISO/IEC/IEEE 29148:2018, Requirements engineering
- Other project documents (add here)

1.5 Overview

Briefly explain how the rest of the document is organized.

2 Overall Description

2.1 Product Perspective

Explain how this product fits into the larger system or environment. Include a system context diagram if useful.

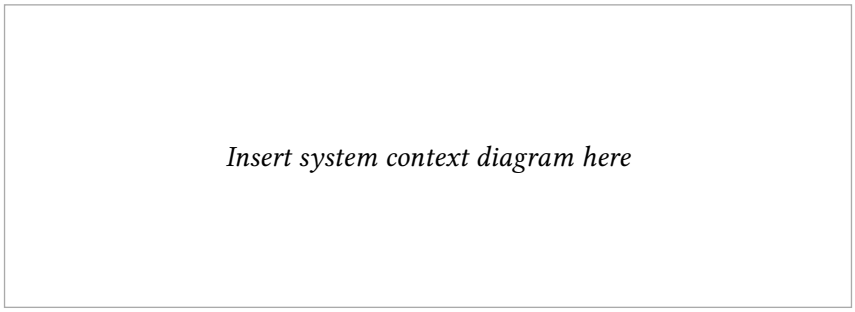


Figure 1: System context diagram

See Figure 1 for the system context.

2.2 Product Functions

Summarize the major functions the software will perform.

2.3 User Classes and Characteristics

User class	Characteristics
Moderator	Manages reviews, new books, Author submition tickets and system configuration.
Author	Has access to the core features of the API, but also indicate which books belong to them.
Standard User	Uses core features of the API.
Logged off User	Uses mainly the listings and views the reviews

2.4 Design and Implementation Constraints

The mandatory constraint is to have an API made with C sharp with dotnet framework, outside of that we were given as much creativity freedom as we wanted.

2.5 Assumptions and Dependencies

Since its for show purpose some parts of the security are going to be ignored, for example csrf protection is one of the parts, not only that but also allowing .env variables to be injected through a .env and not with the c sharp standard.

As for the dependencies, we are currently relying on a few, like Xunit for Unit testing, AspNet Authentication package this allows us to make sign in and sign on's more secure, the DotNetEnv package which allows us to read automatically the .env variables, making the development more smooth and finally the the OpenApi specification package, which makes the API expose an endpoint showing a self-documenting user interface with all the paths the REST API provides.

3 Specific Requirements

3.1 Functional Requirements

3.1.1 User Authentication

ID	Requirement
RF-001	O sistema deve permitir ao utilizador fazer sign up.
RF-002	O sistema deve permitir ao utilizador fazer sign in.
RF-003	O sistema deve permitir ao utilizador fazer logout.
RF-004	O sistema deve permitir ao utilizador recuperar a password.

3.1.2 Data Management

ID	Requirement
RF-005	O sistema deve permitir ao utilizador alterar as suas informações.
RF-006	O sistema deve permitir ao moderador bloquear um utilizador.
RF-007	O sistema deve permitir ao moderador adicionar um livro.
RF-008	O sistema deve permitir ao moderador remover uma review.
RF-009	O sistema deve permitir ao utilizador adicionar um livro há sua biblioteca.
RF-010	O sistema deve permitir ao utilizador remover um livro da sua biblioteca.
RF-011	O sistema deve permitir ao utilizador definir um livro como um desejo (wish).
RF-012	O sistema deve permitir ao utilizador remover um livro de ser um desejo (wish).
RF-013	O sistema deve permitir ao utilizador definir um livro como ‘lido’.
RF-014	O sistema deve permitir ao utilizador definir um livro como ‘para ler’.
RF-015	O sistema deve permitir ao utilizador adicionar uma review a um livro.
RF-016	O sistema deve obrigar ao utilizador dar um rating após efetuar uma review.
RF-017	O sistema deve permitir ao utilizador editar a sua review.
RF-018	O sistema deve permitir ao utilizador alterar o rating da sua review.
RF-019	O sistema deve permitir ao utilizador remover uma review sua.
RF-020	O sistema tem de mostrar quando uma review foi editada.
RF-021	O sistema tem de mostrar quando uma review foi apagada.
RF-022	O sistema deve permitir que a review seja vazia, mas tem de ter rating.
RF-023	O sistema deve permitir a utilizadores submeterem um ticket para serem autores.
RF-024	O sistema deve permitir ao moderador aceitar ou negar um ticket para um utilizador ser autor.

RF-025	O sistema deve permitir ao utilizador indicar, no ticket, os livros de que é autor.
RF-026	O sistema deve permitir ao moderador aceitar ou negar livros na lista do ticket (Ref: Req25).
RF-027	O sistema deve permitir ao utilizador definir uma meta de leitura.
RF-028	O sistema deve permitir ao utilizador ver se está ou não a cumprir com a meta de leitura definida.
RF-029	O sistema deve de enviar um e-mail a congratular o utilizador por ter atingido a sua meta de leitura.

3.2 External Interface Requirements

3.2.1 Software Interfaces

Describe APIs, databases, third-party services, and protocols.

Interface	Description	Protocol
MySql 8.4	Persistent storage for users, books, and reviews. The API reads and writes records on every CRUD request.	SQL over TCP
REST API (provided)	Exposes endpoints for managing books, reviews, user data and for authentication. Consumed by any HTTP client, such as a web or mobile front end.	HTTP, REST,JSON
JWT	Used to authenticate requests. The API issues a signed token on login and validates it on protected endpoints.	HTTP `Authorization: Bearer` header

3.2.2 Communication Interfaces

Aspect	Description	Standard
Transport	All client communication happens over HTTP	HTTP
Message format	Request and response bodis are JSON with the header `Content-Type: application/json`.	JSON
Authentication	Clients send the JWT in the `Authorization` header on every protected endpoint.	Bearer token (JWT)
Error handling	Standard HTTP status codes are used.	HTTP status codes

3.3 Non-Functional Requirements

3.3.1 Security

ID	Requirement
NFR-001	Request to add, modify or delete data shall always have a Bearer token.
NFR-002	Passwords shall be stored as an Hash of the original password

3.3.2 Maintainability

ID	Requirement
NFR-001	The codebase shall have automated test coverage of at least 70%.

4 Domain

This chapter explains the domain of the API being developed, it will discribe the entities and its relationship with the system.

4.1 Entity relationship

Figure 2 shows the output of the conceptualization of the system's data and the relationships between that data.

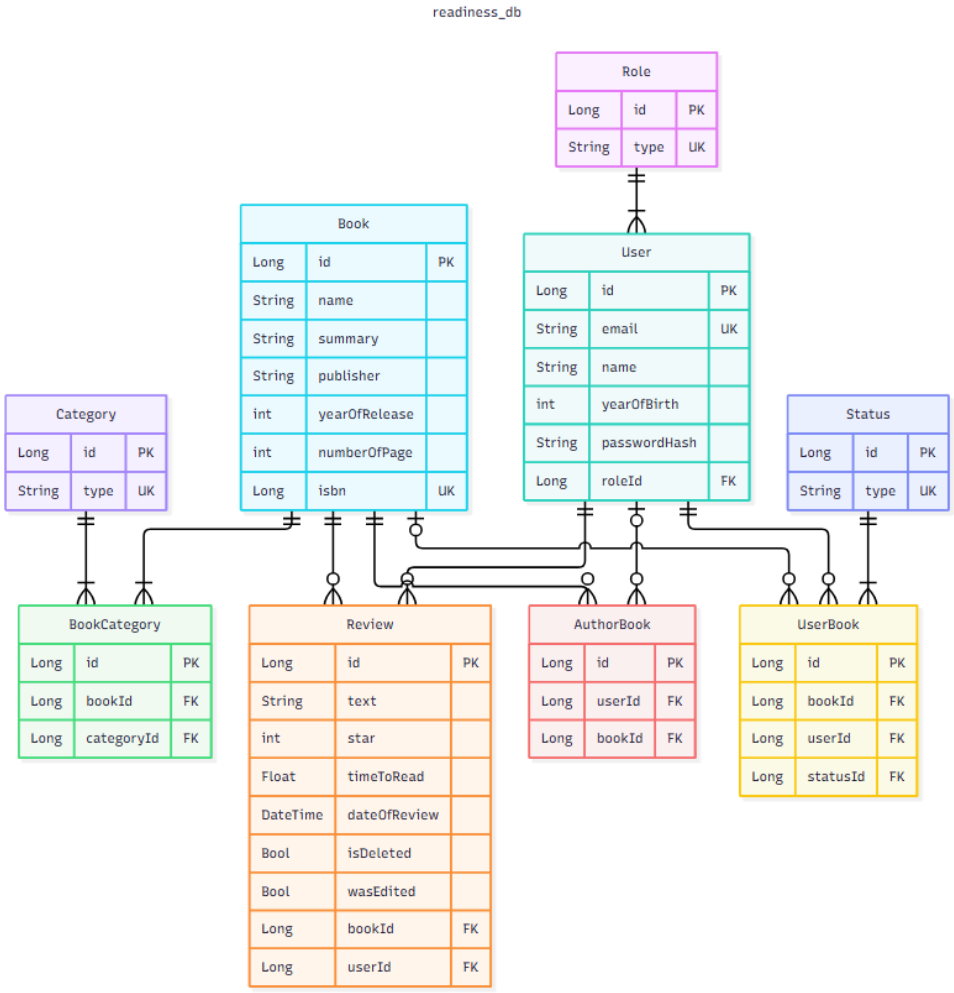


Figure 2: Database ERD diagram

The following chapters describe the data shown on the erd diagram.

4.2 User

The user is the core of the API, he can have multiple roles that define what he can or can't do in the system. In essence, the normal User visualizes and interacts with the whole system, this includes being able to add books onto a whishlist, being also able to add books as being *read* or *to be read*, and also it can add a review of a given book.

If the user has other roles such as moderator role or author role he can have other functionalities on the API. As a moderator he can remove certain reviews, add books into the system and even approve

author role to be given. Finally, an user that is an author can do all the normal actions of the user but can associate his books to himself.

4.3 Role

The Role simply difines what authorizations another user might have on the system. In this case, it has 3 role types, the moderator role, the author role and the user role. The user role is attributed to everyone making its sign up on the system as the default role.

4.4 Book

The Book entity is also one of the core parts of the system. They contain information such as the name of the book, the summary, the publisher of the book, the year of release, the number of pages and the isbn. This entity has a relationship with other major entities, such as the review entity and user.

4.5 Review

5 Appendices

5.1 Appendix A: Use Cases

Add detailed use cases or user stories here.

5.2 Appendix B: Open Issues

ID	Issue	Owner
TBD-1	Describe an unresolved question.	Name